

Nom et Prenom: Lyazali Zakaria

Filiere : Ingénierie des Systèmes D'Information Et Big Data

TP – Bases de Données

TP 1 : Les Vues

TP 2 : Vues et Index

TD 1 : Sous-requêtes

TP 3 : PL/SQL – Blocs et structures de contrôle

TP 4 : PL/SQL – Variables avancées

TP 5 : Procédures et fonctions

TP 6 : Curseurs

TP 7 : Triggers

TP 1 – Les Vues en Bases de Données

Exercice 1 – Vue simple emp_paris

On joint Employe et Departement pour ne garder que les employés à Paris :

```
CREATE VIEW emp_paris AS
SELECT e.emp_id, e.nom, e.salaire
FROM Employe e
JOIN Departement d ON e.dept_id = d.dept_id
WHERE d.ville = 'Paris';

-- Affichage du contenu
SELECT * FROM emp_paris;
```

Exercice 2 – Vue avec agrégations stat_dept

Statistiques salariales par département :

```
CREATE VIEW stat_dept AS
SELECT d.nom_dept,
MIN(e.salaire) AS salaire_min,
MAX(e.salaire) AS salaire_max,
AVG(e.salaire) AS salaire_moyen
FROM Employe e
JOIN Departement d ON e.dept_id = d.dept_id
GROUP BY d.nom_dept;
```

Vue avec GROUP BY : lecture seule, pas d'INSERT/UPDATE possible dessus.

Exercice 3 – WITH CHECK OPTION : emp_lyon

```
CREATE VIEW emp_lyon AS
SELECT e.*
FROM Employe e
JOIN Departement d ON e.dept_id = d.dept_id
WHERE d.ville = 'Lyon'
WITH CHECK OPTION;
```

Toute modification qui ferait sortir la ligne du périmètre 'Lyon' est rejetée (ORA-01402).

Exercice 4 – Vue en lecture seule

```
CREATE VIEW emp_readonly AS
SELECT * FROM Employe
WITH READ ONLY;
```

Exercice 5 – Vue sans salaire (sécurité)

```
CREATE VIEW emp_public AS
SELECT emp_id, nom, dept_id
FROM Employe;
```

Une vue améliore-t-elle les performances ? Non, une vue classique réévalue la requête à chaque appel. Pour les performances on utilise des vues matérialisées qui stockent le résultat physiquement.

Exercice 6 – Analyse globale

1. Architecture de vues pour la sécurité

```
-- Managers : uniquement leur département
CREATE VIEW v_manager AS
SELECT emp_id, nom, dept_id FROM Employe
WHERE dept_id = SYS_CONTEXT('USERENV','CLIENT_INFO');

-- RH : tout
CREATE VIEW v_rh AS SELECT * FROM Employe;

-- Employés : pas les salaires
CREATE VIEW v_employes_pub AS
SELECT emp_id, nom, dept_id FROM Employe;
```

2. Limites sur grand volume

Sur des centaines de milliers de lignes, chaque accès à la vue réévalue la jointure depuis zéro. Sans index, une vue avec GROUP BY ou plusieurs jointures peut être très lente. L'optimiseur n'a pas de statistiques propres à la vue.

3. Vues matérialisées

Une vue matérialisée stocke physiquement le résultat. On peut la rafraîchir ON COMMIT ou périodiquement. Idéale pour des rapports consultés souvent (salaires moyens par département par exemple).

4. Vues vs GRANT/REVOKE

GRANT/REVOKE contrôle l'accès à une table entière. Les vues permettent de restreindre les colonnes ou les lignes visibles. Les deux se complètent.

5. La vue seule suffit-elle pour la sécurité ?

Non. Une vue ne chiffre pas les données, un DBA peut toujours accéder aux tables directement. C'est un outil parmi d'autres dans une stratégie de sécurité.

6. Stratégie d'optimisation

Index sur les colonnes de jointure (dept_id), partitionnement par département pour les très grandes tables, vues matérialisées pour les rapports agrégés fréquents.

TP 2 – Vues et Index

Schéma : EMP(Mat, NomE, Poste, DatEmb, Sup, Salaire, Commission, NumDept) /
DEPT(NumDept, NomDept, Lieu) / PROJET / PARTICIPATION

Exercice 1 – Vues

1. Vue V_EMP

```
CREATE VIEW V_EMP AS
SELECT e.Matr,
       e.NomE,
       e.NumDept,
       (NVL(e.Salaire,0) + NVL(e.Commission,0)) AS GAINS,
       d.Lieu
FROM EMP e
JOIN DEPT d ON e.NumDept = d.NumDept;
```

2. Sélection avec filtre sur GAINS

```
SELECT * FROM V_EMP WHERE GAINS > 10000;
```

3. Mise à jour du nom via la vue

```
UPDATE V_EMP SET NomE = 'MARTIN2' WHERE NomE = 'MARTIN';
```

Cela fonctionne car V_EMP préserve la clé de EMP. Oracle répercute la modification sur la table de base.

4. Vue VEMP10 sans CHECK

```
CREATE VIEW VEMP10 AS
SELECT * FROM EMP WHERE NumDept = 10;

-- Insertion d'un employé du département 20
INSERT INTO VEMP10 (Matr, NomE, Poste, DatEmb, Sup, Salaire, Commission, NumDept)
VALUES (9999, 'SOULIER', 'VENDEUR', SYSDATE, NULL, 2500, NULL, 20);
```

SOULIER est inséré dans EMP avec NumDept=20. Via VEMP10 il est invisible (vue filtrée sur 10). Via EMP il apparaît. C'est le problème de l'insertion fantôme.

5. Recréer VEMP10 avec CHECK

```
DROP VIEW VEMP10;

CREATE VIEW VEMP10 AS
SELECT * FROM EMP
WHERE NumDept = 10
WITH CHECK OPTION;
```

6. Tentative d'insertion hors périmètre

```
INSERT INTO VEMP10 (Matr, NomE, NumDept)
VALUES (8888, 'BALARD', 30);
-- ORA-01402 : view WITH CHECK OPTION where-clause violation
```

Modifier le NumDept d'un employé visible pour le faire passer en dept 30 échoue aussi.

7. Pourcentage du salaire par rapport au total du département

```
-- Avec une vue intermédiaire
CREATE VIEW total_sal_dept AS
SELECT NumDept, SUM(Salaire) AS total_dept
FROM EMP GROUP BY NumDept;
```

```
SELECT e.NomE, e.Salaire,  
ROUND(e.Salaire / t.total_dept * 100, 2) AS pct  
FROM EMP e JOIN total_sal_dept t ON e.NumDept = t.NumDept;
```

```
-- Sans vue (sous-requête dans FROM)  
SELECT e.NomE, e.Salaire,  
ROUND(e.Salaire /  
(SELECT SUM(e2.Salaire) FROM EMP e2  
WHERE e2.NumDept = e.NumDept) * 100, 2) AS pct  
FROM EMP e;
```

Exercice 2 – Index

1. Créer les index

```
CREATE UNIQUE INDEX idx_emp_nome ON EMP (NomE);  
CREATE INDEX idx_emp_dept ON EMP (NumDept);
```

2. Supprimer l'index sur NomE

```
DROP INDEX idx_emp_nome;
```

TD 1 – Sous-requêtes

Exercice 1 – Base ETUDIANTS

Schéma : ETUDIANT(Numetu, Nometu, Dtnaiss, Cdsexe) / SEXE / ENSEIGNANT /
MATIERE(Numat, Nomat, Coeff, Numens) / NOTES(Numetu, Numat, Note)

Q1. Étudiants sans date de naissance

```
SELECT Numetu, Nometu, Cdsexe
FROM ETUDIANT
WHERE Dtnaiss IS NULL;
```

Q2. 2e lettre du nom est 'a'

```
SELECT Nometu, Dtnaiss
FROM ETUDIANT
WHERE Nometu LIKE '_a%';
```

Q3. Matières avec coefficient entre 1 et 2

```
SELECT Nomat FROM MATIERE WHERE Coeff BETWEEN 1 AND 2;
```

Q4. Notes de l'étudiant 1 égales aux notes de l'étudiant 2

```
SELECT n1.Note
FROM NOTES n1
WHERE n1.Numetu = 1
AND n1.Note IN (SELECT n2.Note FROM NOTES n2 WHERE n2.Numetu = 2);
```

Q5. Notes de l'étudiant 1 supérieures à au moins une note de l'étudiant 2

```
SELECT n1.Note
FROM NOTES n1
WHERE n1.Numetu = 1
AND n1.Note > SOME (SELECT n2.Note FROM NOTES n2 WHERE n2.Numetu = 2);
```

Q6. Notes de l'étudiant 1 inférieures à toutes celles de l'étudiant 9

```
SELECT n1.Note
FROM NOTES n1
WHERE n1.Numetu = 1
AND n1.Note < ALL (SELECT n9.Note FROM NOTES n9 WHERE n9.Numetu = 9);
```

Q7. Étudiants sans aucune note

```
SELECT * FROM ETUDIANT e
WHERE NOT EXISTS (SELECT 1 FROM NOTES n WHERE n.Numetu = e.Numetu);
```

Q8. Âge moyen garçons/filles au 01/01/2000

```
SELECT s.Lbsexe,
AVG(FLOOR(MONTHS_BETWEEN(DATE '2000-01-01', e.Dtnaiss) / 12)) AS age_moy
FROM ETUDIANT e JOIN SEXE s ON e.Cdsexe = s.Cdsexe
GROUP BY s.Lbsexe;
```

Q9. Enseignants d'histoire

```
SELECT e.Nomens, e.Grade
FROM ENSEIGNANT e JOIN MATIERE m ON m.Numens = e.Numens
WHERE UPPER(m.Nomat) = 'HISTOIRE';
```

Q10. Étudiants sans notes en Sociologie

```
SELECT Nometu, Numetu FROM ETUDIANT
WHERE Numetu NOT IN (
```

```

SELECT n.Numetu FROM NOTES n JOIN MATIERE m ON n.Numat = m.Numat
WHERE UPPER(m.Nomat) = 'SOCIOLOGIE'
);

```

Exercice 2 – Schéma EMP/DEPT

Q11. Lieux avec commission (sous-requête)

```

SELECT DISTINCT d.Lieu FROM DEPT d
WHERE d.NumDept IN (
SELECT e.NumDept FROM EMP e WHERE e.Commission IS NOT NULL
);

```

Q12. Départements avec au moins un ingénieur

```

SELECT d.NomDept, d.Lieu FROM DEPT d
WHERE d.NumDept IN (
SELECT e.NumDept FROM EMP e WHERE e.Poste = 'INGENIEUR'
);

```

```

-- Version jointure
SELECT DISTINCT d.NomDept, d.Lieu FROM DEPT d
JOIN EMP e ON e.NumDept = d.NumDept WHERE e.Poste = 'INGENIEUR';

```

Q13. Départements sans ingénieur

```

SELECT d.NomDept, d.Lieu FROM DEPT d
WHERE d.NumDept NOT IN (
SELECT e.NumDept FROM EMP e WHERE e.Poste = 'INGENIEUR'
);

```

Q14. Employés gagnant moins de 50% du salaire de leur supérieur

```

SELECT e.NomE, e.Salaire FROM EMP e
WHERE e.Salaire < 0.5 * (
SELECT s.Salaire FROM EMP s WHERE s.Matr = e.Sup
);

```

Q15. Employés gagnant plus que tous les commerciaux

```

SELECT NomE FROM EMP
WHERE Salaire > ALL (
SELECT Salaire FROM EMP WHERE Poste = 'COMMERCIAL'
);

```

Q16. Employés gagnant plus que tous les commerciaux de leur département

```

-- Inclut les depts sans commerciaux (réponse logiquement valable)
SELECT e.NomE FROM EMP e
WHERE e.Salaire > ALL (
SELECT e2.Salaire FROM EMP e2
WHERE e2.Poste = 'COMMERCIAL' AND e2.NumDept = e.NumDept
);

```

```

-- Exclut les depts sans commerciaux
SELECT e.NomE FROM EMP e
WHERE EXISTS (SELECT 1 FROM EMP e2
WHERE e2.Poste='COMMERCIAL' AND e2.NumDept=e.NumDept) AND
e.Salaire > ALL (
SELECT e2.Salaire FROM EMP e2
WHERE e2.Poste='COMMERCIAL' AND e2.NumDept=e.NumDept
);

```

Q17. Même poste et même supérieur que BERGER

```
SELECT NomE FROM EMP
WHERE (Poste, Sup) IN (
SELECT Poste, Sup FROM EMP WHERE NomE = 'BERGER'
)
AND NomE <> 'BERGER';
```

TP 3 – PL/SQL : Blocs, Variables et Structures de contrôle

Exercice 1 – Insérer 1 à 100 dans une table

```
CREATE TABLE Nombre (n NUMBER);

BEGIN
FOR i IN 1..100 LOOP
INSERT INTO Nombre VALUES (i);
END LOOP;
COMMIT;
END;
/
```

Exercice 2 – Somme de 1 à 1000

```
DECLARE
v_somme NUMBER := 0;
BEGIN
FOR i IN 1..1000 LOOP
v_somme := v_somme + i;
END LOOP;
DBMS_OUTPUT.PUT_LINE('Somme = ' || v_somme);
END;
/
```

Résultat : 500 500

Exercice 3 – Nombres pairs entre 1 et 50

```
BEGIN
FOR i IN 1..50 LOOP
IF MOD(i, 2) = 0 THEN
DBMS_OUTPUT.PUT_LINE(i);
END IF;
END LOOP;
END;
/
```

Exercice 4 – Factorielle

```
DECLARE
v_n NUMBER := 7;
v_fact NUMBER := 1;
BEGIN
FOR i IN 1..v_n LOOP
v_fact := v_fact * i;
END LOOP;
DBMS_OUTPUT.PUT_LINE(v_n || ' ! = ' || v_fact);
END;
/
```

Exercice 5 – Insérer 10 étudiants

```
CREATE TABLE Etudiant (id NUMBER, nom VARCHAR2(30), note NUMBER);

BEGIN
FOR i IN 1..10 LOOP
```

```

INSERT INTO Etudiant VALUES (i, 'Etudiant_' || i,
ROUND (DBMS_RANDOM.VALUE (5,20)));
END LOOP;
COMMIT;
END;
/

```

Exercice 6 – Boucle WHILE de 10 à 1

```

DECLARE
v_i NUMBER := 10;
BEGIN
WHILE v_i >= 1 LOOP
DBMS_OUTPUT.PUT_LINE(v_i);
v_i := v_i - 1;
END LOOP;
END;
/

```

Exercice 7 – Augmenter les prix de 10%

```

BEGIN
UPDATE Produit SET prix = prix * 1.10;
COMMIT;
DBMS_OUTPUT.PUT_LINE(SQL%ROWCOUNT || ' produit(s) mis à jour.');
```

```

END;
/

```

Exercice 8 – Supprimer produits sous 50

```

BEGIN
DELETE FROM Produit WHERE prix < 50;
COMMIT;
DBMS_OUTPUT.PUT_LINE(SQL%ROWCOUNT || ' produit(s) supprimé(s).');
```

```

END;
/

```

Exercice 9 – Test positif / négatif / nul

```

DECLARE
v_n NUMBER := -3;
BEGIN
IF v_n > 0 THEN
DBMS_OUTPUT.PUT_LINE(v_n || ' est positif.');
```

```

ELSIF v_n < 0 THEN
DBMS_OUTPUT.PUT_LINE(v_n || ' est négatif.');
```

```

ELSE
DBMS_OUTPUT.PUT_LINE('Le nombre est nul.');
```

```

END IF;
END;
/

```

Exercice 10 – Carrés de 1 à 20

```

CREATE TABLE Carre (nombre NUMBER, carre NUMBER);

BEGIN
FOR i IN 1..20 LOOP

```

```
INSERT INTO Carre VALUES (i, i*i);  
END LOOP;  
COMMIT;  
END;  
/
```

TP 4 – Variables avancées : %TYPE, %ROWTYPE, RECORD, Collections

Exercice 1 – Variables scalaires

```
SET SERVEROUTPUT ON;
DECLARE
v_nom VARCHAR2(30) := 'Alice';
v_salaire NUMBER(8,2) := 3500.50;
v_date_emb DATE := SYSDATE;
c_taux CONSTANT NUMBER(3,2) := 0.15;
v_prime NUMBER(8,2);
BEGIN
v_prime := v_salaire * c_taux;
DBMS_OUTPUT.PUT_LINE('Employé : ' || v_nom);
DBMS_OUTPUT.PUT_LINE('Prime : ' || v_prime);
END;
/
```

Une constante doit être initialisée à la déclaration. NOT NULL impose aussi une valeur initiale. BOOLEAN existe en PL/SQL mais ne s'utilise pas directement dans SQL.

Exercice 2 – Portée des variables

```
DECLARE
v_externe NUMBER := 10;
BEGIN
DECLARE
v_interne NUMBER := 20;
BEGIN
DBMS_OUTPUT.PUT_LINE('Interne : ' || v_interne);
DBMS_OUTPUT.PUT_LINE('Externe : ' || v_externe);
END;
-- v_interne n'est plus accessible ici
END;
/
```

Exercice 3 – %TYPE

```
DECLARE
v_nom employees.nom%TYPE;
v_salaire employees.salaire%TYPE;
BEGIN
v_nom := 'Jean';
v_salaire := 5000;
DBMS_OUTPUT.PUT_LINE(v_nom || ' gagne ' || v_salaire);
END;
/
```

%TYPE adapte le type automatiquement si la colonne change. Pratique pour la maintenance.

Exercice 4 – %ROWTYPE

```
DECLARE
v_emp employees%ROWTYPE;
BEGIN
v_emp.id := 1;
v_emp.nom := 'Maria';
```

```

v_emp.salaire := 6000;
DBMS_OUTPUT.PUT_LINE(v_emp.nom || ' - ' || v_emp.salaire);
END;
/

```

Exercice 5 – Collection (table associative)

```

DECLARE
TYPE table_salaires IS TABLE OF NUMBER INDEX BY BINARY_INTEGER;
v_salaires table_salaires;
BEGIN
v_salaires(1) := 3000;
v_salaires(2) := 4500;
v_salaires(3) := 5000;
FOR i IN 1..3 LOOP
DBMS_OUTPUT.PUT_LINE('Salaire : ' || v_salaires(i));
END LOOP;
END;
/

```

Exercice intégré – RECORD + Collection + %ROWTYPE

```

DECLARE
TYPE table_emp IS TABLE OF employes%ROWTYPE INDEX BY BINARY_INTEGER;
v_emps table_emp;
BEGIN
FOR i IN 1..3 LOOP
v_emps(i).id := i;
v_emps(i).nom := 'Employe_' || i;
v_emps(i).salaire := 3000 + i * 1000;
END LOOP;
FOR i IN 1..3 LOOP
DBMS_OUTPUT.PUT_LINE(v_emps(i).nom || ' gagne ' || v_emps(i).salaire);
END LOOP;
END;
/

```

Exercice final de synthèse

```

DECLARE
TYPE t_employe IS RECORD (
id NUMBER,
nom VARCHAR2(50),
salaire NUMBER(8,2)
);
TYPE t_table IS TABLE OF t_employe INDEX BY BINARY_INTEGER;
v_tab t_table;
v_somme NUMBER := 0;
v_max NUMBER := 0;
BEGIN
FOR i IN 1..5 LOOP
v_tab(i).id := i;
v_tab(i).nom := 'Emp_' || i;
v_tab(i).salaire := 2000 + i * 500;
END LOOP;

FOR i IN 1..5 LOOP

```

```
v_somme := v_somme + v_tab(i).salaire;  
IF v_tab(i).salaire > v_max THEN  
v_max := v_tab(i).salaire;  
END IF;  
END LOOP;  
  
DBMS_OUTPUT.PUT_LINE('Salaire moyen : ' || v_somme / 5);  
DBMS_OUTPUT.PUT_LINE('Salaire max : ' || v_max);  
END;  
/
```

TP 5 – Procédures stockées et fonctions PL/SQL

Table utilisée : EMPLOYES(ENUM, ENAME, SALARY, ADDRESS, DEPT)

Exercice 1 – Bloc Hello et fonction carré

```
-- Bloc anonyme
BEGIN
  DBMS_OUTPUT.PUT_LINE('Hello');
END;
/

-- Fonction carré
CREATE OR REPLACE FUNCTION carre(p_n IN NUMBER) RETURN NUMBER IS
BEGIN
  RETURN p_n * p_n;
END;
/

SELECT carre(7) FROM DUAL;
```

Si on omet RETURN dans une fonction, Oracle compile mais lève une erreur à l'exécution. Une procédure peut renvoyer des valeurs via des paramètres OUT mais ne s'utilise pas dans une requête SQL, contrairement à une fonction. DUAL est une table virtuelle d'une ligne pour tester des expressions.

Exercice 2 – Maximum de deux nombres (OUT)

```
CREATE OR REPLACE PROCEDURE max_deux(
  p_a IN NUMBER,
  p_b IN NUMBER,
  p_max OUT NUMBER
) IS
BEGIN
  IF p_a >= p_b THEN p_max := p_a;
  ELSE p_max := p_b;
  END IF;
END;
/

DECLARE v_res NUMBER;
BEGIN
  max_deux(12, 7, v_res);
  DBMS_OUTPUT.PUT_LINE('Max = ' || v_res);
END;
/
```

Un paramètre OUT non affecté dans la procédure reste NULL sans erreur automatique. Portée : le paramètre OUT n'existe que le temps de l'appel.

```
-- Version fonction
CREATE OR REPLACE FUNCTION max_deux_f(p_a IN NUMBER, p_b IN NUMBER)
RETURN NUMBER IS
BEGIN
  RETURN GREATEST(p_a, p_b);
END;
/
```

Exercice 3 – Permutation (IN OUT)

```
CREATE OR REPLACE PROCEDURE permuter(
  p_a IN OUT NUMBER,
  p_b IN OUT NUMBER
) IS
  v_tmp NUMBER;
BEGIN
  v_tmp := p_a;
  p_a := p_b;
  p_b := v_tmp;
END;
/

DECLARE
  v_x NUMBER := 10;
  v_y NUMBER := 20;
BEGIN
  permuter(v_x, v_y);
  DBMS_OUTPUT.PUT_LINE('x=' || v_x || ' y=' || v_y);
END;
/
```

Avec IN uniquement, on ne pourrait pas modifier les variables en dehors : erreur de compilation. IN OUT permet de lire et d'écrire la même variable.

Exercice 4 – Augmenter le salaire

```
CREATE OR REPLACE PROCEDURE augmenter_salaire(
  p_nom IN VARCHAR2,
  p_taux IN NUMBER
) IS
BEGIN
  UPDATE EMPLOYES
  SET SALARY = SALARY * (1 + p_taux / 100)
  WHERE ENAME = p_nom;

  IF SQL%ROWCOUNT = 0 THEN
    DBMS_OUTPUT.PUT_LINE('Employé introuvable : ' || p_nom);
  ELSE
    DBMS_OUTPUT.PUT_LINE(SQL%ROWCOUNT || ' ligne(s) mise(s) à jour.');
```

Si plusieurs employés ont le même nom, toutes les lignes sont mises à jour. Passer l'ENUM (clé primaire) garantit une seule ligne. SQL%ROWCOUNT vaut le nombre de lignes affectées par le dernier DML.

```
-- Variante avec ENUM
CREATE OR REPLACE PROCEDURE augmenter_salaire_enum(
  p_enum IN VARCHAR2, p_taux IN NUMBER
) IS
BEGIN
  UPDATE EMPLOYES SET SALARY = SALARY * (1 + p_taux/100) WHERE ENUM = p_enum;
END;
/
```

Exercice 5 – Conversion dirhams vers euros

```
CREATE OR REPLACE FUNCTION dirham_vers_euro(p_montant IN NUMBER)
RETURN NUMBER IS
c_taux CONSTANT NUMBER := 11;
BEGIN
RETURN ROUND(p_montant / c_taux, 2);
END;
/

SELECT ENAME, SALARY, dirham_vers_euro(SALARY) AS salary_eur FROM EMPLOYES;
```

Une fonction ne doit pas contenir de COMMIT/ROLLBACK quand elle est appelée depuis SQL (ORA-14552). Version avec taux variable :

```
CREATE OR REPLACE FUNCTION dirham_vers_euro(
p_montant IN NUMBER,
p_taux IN NUMBER DEFAULT 11
) RETURN NUMBER IS
BEGIN
RETURN ROUND(p_montant / p_taux, 2);
END;
/
```

Exercice 6 – Employés au-dessus d'un seuil

```
CREATE OR REPLACE PROCEDURE afficher_hauts_salaires(p_seuil IN NUMBER) IS BEGIN
FOR emp IN (SELECT ENAME, SALARY FROM EMPLOYES
WHERE SALARY > p_seuil ORDER BY SALARY DESC) LOOP
DBMS_OUTPUT.PUT_LINE(emp.ENAME || ' : ' || emp.SALARY);
END LOOP;
END;
/
```

On peut aussi faire la même chose sans procédure, avec un curseur FOR directement dans un bloc anonyme. Ajouter un paramètre de tri : passer 'ASC' ou 'DESC' en VARCHAR2 et construire la requête dynamiquement avec EXECUTE IMMEDIATE.

Exercice 7 – Salaire moyen par département

```
CREATE OR REPLACE FUNCTION salaire_moyen_dept(p_dept IN VARCHAR2)
RETURN NUMBER IS
v_moy NUMBER;
BEGIN
SELECT AVG(SALARY) INTO v_moy FROM EMPLOYES WHERE DEPT = p_dept;
RETURN NVL(v_moy, 0);
EXCEPTION
WHEN NO_DATA_FOUND THEN RETURN NULL;
END;
/
```

NVL protège contre NULL quand le département existe mais n'a pas d'employés. On peut appeler cette fonction dans une procédure qui met à jour une table de statistiques :

```
UPDATE stats_dept SET salaire_moy = salaire_moyen_dept(dept_id);
```

Exercice 8 – Mise à jour d'adresse par nom

```

CREATE OR REPLACE PROCEDURE maj_adresse(
p_nom IN VARCHAR2,
p_adresse IN VARCHAR2
) IS
BEGIN
IF p_adresse IS NULL OR LENGTH(TRIM(p_adresse)) = 0 THEN
RAISE_APPLICATION_ERROR(-20100, 'Adresse vide non autorisée.');
```

```

END IF;

UPDATE EMPLOYEES SET ADDRESS = p_adresse WHERE ENAME = p_nom;

IF SQL%ROWCOUNT = 0 THEN
DBMS_OUTPUT.PUT_LINE('Employé introuvable : ' || p_nom); END
IF;
END;
/
```

Pour garantir une seule mise à jour quand le nom n'est pas unique, il vaut mieux passer l'ENUM. La vérification de l'adresse non vide est faite avant l'UPDATE.

Exercice 9 – Nombre d'employés par département

```

CREATE OR REPLACE FUNCTION nb_employes_dept(p_dept IN VARCHAR2)
RETURN NUMBER IS
v_cnt NUMBER;
BEGIN
SELECT COUNT(*) INTO v_cnt FROM EMPLOYEES WHERE DEPT = p_dept;
RETURN v_cnt;
END;
/
```

COUNT(*) compte toutes les lignes. COUNT(ENUM) ne compte que les lignes où ENUM est non NULL — sur une clé primaire la différence est nulle.

Exercice 10 – Suppression des petits salaires

```

CREATE OR REPLACE PROCEDURE suppr_bas_salaires(p_seuil IN NUMBER) IS
BEGIN
SAVEPOINT avant_suppression;

DELETE FROM EMPLOYEES WHERE SALARY < p_seuil;
DBMS_OUTPUT.PUT_LINE(SQL%ROWCOUNT || ' employé(s) supprimé(s).');
```

```

-- Pas de COMMIT ici : laissé à la transaction appelante
EXCEPTION
WHEN OTHERS THEN
ROLLBACK TO avant_suppression;
RAISE;
END;
/
```

COMMIT dans une procédure appelée par un programme externe peut valider des modifications partielles. SAVEPOINT permet d'annuler seulement ce bloc en cas d'erreur tout en laissant le reste de la transaction intact.

Tableau récapitulatif procédure / fonction :

Critère	Procédure	Fonction
-----	-----	-----

Retourne une valeur | Non (OUT possible) | Oui (RETURN obligatoire)
Appel dans une requête | Non | Oui
Transaction (COMMIT) | Possible (déconseillé) | Interdit en SQL
Utilisation typique | Action / traitement | Calcul / transformation

TP 6 – Curseurs PL/SQL

Un curseur est une zone mémoire qui pointe sur la ligne courante d'un résultat SELECT. OPEN charge le résultat, FETCH récupère une ligne, CLOSE libère la mémoire.

Exercice 1 – Curseur explicite (base Usine)

Schéma : Commande(Numcmd, Datcmd, Numfour, Mncmd) / Fournisseurs(Numfour, Adrfour)

Q1. Liste de toutes les commandes

```
DECLARE
CURSOR c_cmd IS SELECT Numcmd, Datcmd, Numfour, Mncmd FROM Commande;
v_cmd c_cmd%ROWTYPE;
BEGIN
OPEN c_cmd;
LOOP
FETCH c_cmd INTO v_cmd;
EXIT WHEN c_cmd%NOTFOUND;
DBMS_OUTPUT.PUT_LINE(v_cmd.Numcmd || ' ' || v_cmd.Datcmd || '
' || v_cmd.Numfour || ' ' || v_cmd.Mncmd);
END LOOP;
CLOSE c_cmd;
END;
/
```

Q2. Commandes dont le montant dépasse la moyenne

```
DECLARE
v_moy NUMBER;
CURSOR c_cmd IS
SELECT Numcmd, Mncmd FROM Commande WHERE Mncmd > v_moy;
v_cmd c_cmd%ROWTYPE;
BEGIN
SELECT AVG(Mncmd) INTO v_moy FROM Commande;
OPEN c_cmd;
LOOP
FETCH c_cmd INTO v_cmd;
EXIT WHEN c_cmd%NOTFOUND;
DBMS_OUTPUT.PUT_LINE(v_cmd.Numcmd || ' : ' || v_cmd.Mncmd);
END LOOP;
CLOSE c_cmd;
END;
/
```

Q3. Commandes avec fournisseur parisien

```
DECLARE
CURSOR c_paris IS
SELECT c.Numcmd, c.Mncmd
FROM Commande c JOIN Fournisseurs f ON c.Numfour = f.Numfour
WHERE f.Adrfour = 'Paris';
v_r c_paris%ROWTYPE;
BEGIN
OPEN c_paris;
LOOP
FETCH c_paris INTO v_r;
EXIT WHEN c_paris%NOTFOUND;
DBMS_OUTPUT.PUT_LINE(v_r.Numcmd || ' - ' || v_r.Mncmd);
END LOOP;
END;
```

```

END LOOP;
CLOSE c_paris;
END;
/

```

Exercice 2 – Curseur FOR (Gavasoft)

Schéma : Emp(NumE, NomE, Fonction, NumS, Embauche, Salaire, Comm, NumD) / Dept(NumD, NomD, Lieu)

Q1. Employés avec commission, par commission décroissante

```

BEGIN
FOR emp IN (SELECT NomE, Comm FROM Emp
WHERE Comm IS NOT NULL ORDER BY Comm DESC) LOOP
DBMS_OUTPUT.PUT_LINE(emp.NomE || ' : ' || emp.Comm);
END LOOP;
END;
/

```

Q2. Embauchés depuis le 01/09/2006

```

BEGIN
FOR emp IN (SELECT NomE, Embauche FROM Emp
WHERE Embauche >= DATE '2006-09-01') LOOP
DBMS_OUTPUT.PUT_LINE(emp.NomE || ' - ' || emp.Embauche); END
LOOP;
END;
/

```

Q3. Employés travaillant à Créteil

```

BEGIN
FOR emp IN (SELECT e.NomE
FROM Emp e JOIN Dept d ON e.NumD = d.NumD
WHERE d.Lieu = 'Créteil') LOOP
DBMS_OUTPUT.PUT_LINE(emp.NomE);
END LOOP;
END;
/

```

Q4. Subordonnés de Guimezanes

```

DECLARE
v_nums Emp.NumS%TYPE;
BEGIN
SELECT NumE INTO v_nums FROM Emp WHERE NomE = 'Guimezanes';
FOR emp IN (SELECT NomE FROM Emp WHERE NumS = v_nums) LOOP
DBMS_OUTPUT.PUT_LINE(emp.NomE);
END LOOP;
END;
/

```

Q5. Moyenne des salaires

```

DECLARE
v_moy NUMBER;
BEGIN
SELECT AVG(Salaire) INTO v_moy FROM Emp;
DBMS_OUTPUT.PUT_LINE('Moyenne : ' || ROUND(v_moy, 2));
END;

```

/

Q6. Nombre de commissions non NULL

```
DECLARE
v_cnt NUMBER;
BEGIN
SELECT COUNT(Comm) INTO v_cnt FROM Emp WHERE Comm IS NOT NULL;
DBMS_OUTPUT.PUT_LINE('Commissions non NULL : ' || v_cnt);
END;
/
```

Q7. Employés gagnant plus que la moyenne

```
DECLARE
v_moy NUMBER;
BEGIN
SELECT AVG(Salaire) INTO v_moy FROM Emp;
FOR emp IN (SELECT NomE, Salaire FROM Emp WHERE Salaire > v_moy) LOOP
DBMS_OUTPUT.PUT_LINE(emp.NomE || ' : ' || emp.Salaire);
END LOOP;
END;
/
```

TP 7 – Triggers PL/SQL

Un trigger est un bloc PL/SQL exécuté automatiquement sur un événement DML (INSERT, UPDATE, DELETE). BEFORE pour bloquer, AFTER pour réagir. FOR EACH ROW pour traiter ligne par ligne. :NEW contient les nouvelles valeurs, :OLD les anciennes.

Règle 1 – Interdire la modification de note_min dans prerequis

```
CREATE OR REPLACE TRIGGER trg_prerequis_no_update_note_min
BEFORE UPDATE OF note_min ON prerequis
FOR EACH ROW
BEGIN
  RAISE_APPLICATION_ERROR(-20001,
    'Modification de note_min dans prerequis interdite.');
```

Règle 2 – Vérifier effectMax à l'inscription

```
CREATE OR REPLACE TRIGGER trg_inscription_check_effectif
BEFORE INSERT ON inscription
FOR EACH ROW
DECLARE
  v_inscrits NUMBER;
  v_effecMax NUMBER;
BEGIN
  SELECT effectMax INTO v_effecMax
  FROM module WHERE code_module = :NEW.code_module;

  SELECT COUNT(*) INTO v_inscrits
  FROM inscription WHERE code_module = :NEW.code_module;

  IF v_inscrits >= v_effecMax THEN
    RAISE_APPLICATION_ERROR(-20002,
      'Module ' || :NEW.code_module || ' a atteint son effectif maximum.');
```

Règle 3 – Créer un examen uniquement si le module a des inscrits

```
CREATE OR REPLACE TRIGGER trg_examen_check_inscrits
BEFORE INSERT ON examen
FOR EACH ROW
DECLARE
  v_nb NUMBER;
BEGIN
  SELECT COUNT(*) INTO v_nb
  FROM inscription WHERE code_module = :NEW.code_module;

  IF v_nb = 0 THEN
    RAISE_APPLICATION_ERROR(-20003,
      'Impossible de créer un examen : aucun inscrit pour ' || :NEW.code_module);
```

Règle 4 – Date d'examen postérieure à l'inscription

```
CREATE OR REPLACE TRIGGER trg_passer_check_date_inscription
BEFORE INSERT ON passer
FOR EACH ROW
DECLARE
v_date_insc DATE;
v_date_exam DATE;
BEGIN
SELECT date_inscription INTO v_date_insc
FROM inscription
WHERE id_etudiant = :NEW.id_etudiant
AND code_module = (SELECT code_module FROM examen
WHERE id_examen = :NEW.id_examen);

SELECT date_examen INTO v_date_exam
FROM examen WHERE id_examen = :NEW.id_examen;

IF v_date_insc >= v_date_exam THEN
RAISE_APPLICATION_ERROR(-20004,
'Inscription postérieure ou égale à la date d'examen.');
```

```
END IF;
END;
/
```

Règle 5 – Détection de circuit dans les prérequis

```
CREATE OR REPLACE FUNCTION existe_circuit(
p_module IN VARCHAR2, p_cible IN VARCHAR2
) RETURN BOOLEAN IS
CURSOR c_enfants IS
SELECT prerequis FROM prerequis WHERE module = p_module;
BEGIN
FOR enfant IN c_enfants LOOP
IF enfant.prerequis = p_cible THEN RETURN TRUE; END IF;
IF existe_circuit(enfant.prerequis, p_cible) THEN RETURN TRUE; END IF;
END LOOP;
RETURN FALSE;
END;
/
```

```
CREATE OR REPLACE TRIGGER trg_prerequis_no_circuit
BEFORE INSERT OR UPDATE ON prerequis
FOR EACH ROW
BEGIN
IF existe_circuit(:NEW.prerequis, :NEW.module) THEN
RAISE_APPLICATION_ERROR(-20005,
'Circuit détecté : ' || :NEW.module || ' -> ' || :NEW.prerequis);
END IF;
END;
/
```

Règle 6 – Vérifier les prérequis à l'inscription

```
CREATE OR REPLACE TRIGGER trg_inscription_check_prerequis
BEFORE INSERT ON inscription
FOR EACH ROW
DECLARE
```

```

CURSOR c_prereq IS
SELECT prerequis, note_min FROM prerequis
WHERE module = :NEW.code_module;
v_note NUMBER;
BEGIN
FOR prereq IN c_prereq LOOP
BEGIN
SELECT MAX(pa.note) INTO v_note
FROM passer pa JOIN examen e ON pa.id_examen = e.id_examen
WHERE pa.id_etudiant = :NEW.id_etudiant
AND e.code_module = prereq.prerequis;
EXCEPTION WHEN NO_DATA_FOUND THEN v_note := NULL;
END;

IF v_note IS NULL OR v_note < prereq.note_min THEN
RAISE_APPLICATION_ERROR(-20006,
'Prerequis ' || prereq.prerequis || ' non satisfait. Note requise : '
|| prereq.note_min);
END IF;
END LOOP;
END;
/

```

Règle 7 – Mise à jour automatique de la moyenne

```

CREATE OR REPLACE TRIGGER trg_passer_update_moyenne
AFTER INSERT OR UPDATE OR DELETE ON passer
FOR EACH ROW
DECLARE
v_id_etudiant NUMBER;
v_moyenne NUMBER;
v_compteur NUMBER;
BEGIN
IF INSERTING OR UPDATING THEN
v_id_etudiant := :NEW.id_etudiant;
ELSE
v_id_etudiant := :OLD.id_etudiant;
END IF;

SELECT SUM(pa.note * e.coefficient) / SUM(e.coefficient), COUNT(*)
INTO v_moyenne, v_compteur
FROM passer pa JOIN examen e ON pa.id_examen = e.id_examen WHERE
pa.id_etudiant = v_id_etudiant AND pa.note IS NOT NULL;

IF v_compteur > 0 THEN
UPDATE etudiant SET moyenne = v_moyenne WHERE id_etudiant = v_id_etudiant; ELSE
UPDATE etudiant SET moyenne = NULL WHERE id_etudiant = v_id_etudiant; END
IF;
END;
/

```

Règle 8 – Modification note_min interdite si elle invalide des inscriptions

```

CREATE OR REPLACE TRIGGER trg_prerequis_check_note_min_update
BEFORE UPDATE OF note_min ON prerequis
FOR EACH ROW

```

```

DECLARE
CURSOR c_etu IS
SELECT DISTINCT id_etudiant FROM inscription
WHERE code_module = :NEW.module;
v_note_max NUMBER;
BEGIN
FOR etu IN c_etu LOOP
SELECT MAX(pa.note) INTO v_note_max
FROM passer pa JOIN examen e ON pa.id_examen = e.id_examen
WHERE pa.id_etudiant = etu.id_etudiant
AND e.code_module = :NEW.prerequis;

IF v_note_max IS NULL OR v_note_max < :NEW.note_min THEN
RAISE_APPLICATION_ERROR(-20008,
'La nouvelle note_min (' || :NEW.note_min ||
') invaliderait l''étudiant ' || etu.id_etudiant);
END IF;
END LOOP;
END;
/

```

Règle 9 – Modification de effecMax sans invalider les inscriptions

```

CREATE OR REPLACE TRIGGER trg_module_check_effecMax_update
BEFORE UPDATE OF effecMax ON module
FOR EACH ROW
DECLARE
v_inscrits NUMBER;
BEGIN
SELECT COUNT(*) INTO v_inscrits
FROM inscription WHERE code_module = :NEW.code_module;

IF :NEW.effecMax < v_inscrits THEN
RAISE_APPLICATION_ERROR(-20009,
'Le nouvel effectif max (' || :NEW.effecMax ||
') est inférieur au nombre d''inscrits (' || v_inscrits || ')'); END
IF;
END;
/

```

Exercices supplémentaires

Contrainte 10 – Pas de réinscription à un module déjà validé

```

CREATE OR REPLACE TRIGGER trg_no_reinscription_valide
BEFORE INSERT ON inscription
FOR EACH ROW
DECLARE
v_note_max NUMBER;
v_note_min NUMBER;
BEGIN
SELECT note_min INTO v_note_min FROM module WHERE code_module = :NEW.code_module;

SELECT MAX(pa.note) INTO v_note_max
FROM passer pa JOIN examen e ON pa.id_examen = e.id_examen
WHERE pa.id_etudiant = :NEW.id_etudiant AND e.code_module = :NEW.code_module;

```

```

IF v_note_max IS NOT NULL AND v_note_max >= v_note_min THEN
RAISE_APPLICATION_ERROR(-20010, 'Module déjà validé – réinscription refusée.');
```

END IF;

END;

/

Contrainte 11 – Pas d'examen à ± 7 jours d'un autre pour le même module

```

CREATE OR REPLACE TRIGGER trg_examen_no_conflit BEFORE
INSERT ON examen
FOR EACH ROW
DECLARE
v_cnt NUMBER;
BEGIN
SELECT COUNT(*) INTO v_cnt FROM examen
WHERE code_module = :NEW.code_module
AND ABS(:NEW.date_examen - date_examen) <= 7;

IF v_cnt > 0 THEN
RAISE_APPLICATION_ERROR(-20011,
'Un examen existe déjà dans les 7 jours pour ce module.');
```

END IF;

END;

/

Contrainte 12 – Moyenne seulement si tous les examens sont passés

```

CREATE TABLE log_moyenne_erreurs (
id_etudiant NUMBER,
message VARCHAR2(200),
dt DATE DEFAULT SYSDATE
);

CREATE OR REPLACE TRIGGER trg_moyenne_complete
AFTER INSERT OR UPDATE OR DELETE ON passer
FOR EACH ROW
DECLARE
v_id NUMBER;
v_inscrits NUMBER;
v_passes NUMBER;
v_moy NUMBER;
BEGIN
v_id := CASE WHEN DELETING THEN :OLD.id_etudiant ELSE :NEW.id_etudiant END;

SELECT COUNT(DISTINCT code_module) INTO v_inscrits
FROM inscription WHERE id_etudiant = v_id;

SELECT COUNT(DISTINCT e.code_module) INTO v_passes
FROM passer pa JOIN examen e ON pa.id_examen = e.id_examen
WHERE pa.id_etudiant = v_id;

IF v_passes = v_inscrits AND v_inscrits > 0 THEN
SELECT SUM(pa.note * e.coefficient) / SUM(e.coefficient) INTO v_moy
FROM passer pa JOIN examen e ON pa.id_examen = e.id_examen
WHERE pa.id_etudiant = v_id AND pa.note IS NOT NULL;
UPDATE etudiant SET moyenne = v_moy WHERE id_etudiant = v_id;
ELSE
UPDATE etudiant SET moyenne = NULL WHERE id_etudiant = v_id;
```

```
INSERT INTO log_moyenne_erreurs(id_etudiant, message)
VALUES (v_id, 'Moyenne non calculée : examens incomplets.');
```

END;
IF;
END;
/

Contrainte 13 – Interdire la suppression d'un module prérequis

```
CREATE OR REPLACE TRIGGER trg_module_no_delete_if_prerequis
BEFORE DELETE ON module
FOR EACH ROW
DECLARE
v_cnt NUMBER;
BEGIN
SELECT COUNT(*) INTO v_cnt
FROM prerequis WHERE prerequis = :OLD.code_module;

IF v_cnt > 0 THEN
RAISE_APPLICATION_ERROR(-20013,
'Impossible de supprimer ' || :OLD.code_module ||
' : utilisé comme prérequis par ' || v_cnt || ' module(s).');
END IF;
END;
/
```
